Programa:	Audio MP3 Maker 1.06
Protección:	Name / Serial
Objetivo:	Registrarnos con cualquier nombre
Dificultad:	Fácil, va bien para empezar
Dowload:	No lo sé
Herramientas:	SoftIce, w32dasm y UltraEdit-32
Cracker:	SyXe'05 [hAcK-c0d3d] Fecha: 12-11-04

Si ejecutamos el programa nos encontramos con una recibida tipo nagscreen que podemos "esquivar" durante 30 días, pasados los cuales el programa deja de funcionar para mal nuestro, además si entramo nos da la opción de introducir un nombre y un número serial que por su puesto no sabemos claro!

Introducimos unos datos cualquiera para dicho nombre y serial y ¡bouuuum!... nos encontramos con un mensaje de error que dice 'Invalid reg code', mirémoslo por el lado bueno, este será nuestro punto de ataque ;D

Abrimos w32dasm y desensamblamos el programa sin dificultad alguna, lo hacemos libremente y con total impunidad, el programador no se ha esmerado en empaquetar el ejecutable ni encriptarlo, ni nada de nada, pues mejor para nosotros, a lo nuestro.. ;)

Una vez desensamblado buscamos en las frases del programa la del mensaje de error que vimos al introducir los datros de registro y lo encontramos, sólo sale una vez en el código, justamente en :0040DA61 y un poco antes está el mensaje de felicitación, y aún más arriba un call que controla todo el rollo este, que si nos fijamos queremos que devuelva eax=0 para que vaya todo bien, pues de no ser así iremos de cabeza al mensaje de error. Pues ya tenemos una cosa clara, el call 00417A2D debe devolver eax valiendo cero, estudiemos cómo trabaja nuestra llamada y obliguémosla a que devuelva eax valiendo cero ;)

```
:0040DA36 E8F29F0000        call 00417A2D      /* llamada interesante */
:0040DA3B 83C408            add esp, 00000008
:0040DA3E 85C0              test eax, eax      ;¿Es eax=0?
:0040DA40 6A00              push 00000000

* Possible StringData Ref from Data Obj ->"Message"
                  |
:0040DA42 68BC7B4400        push 00447BBC
:0040DA47 7518              jne 0040DA61       ;salto MALO, saltará si eax ≠ 0!!

* Possible StringData Ref from Data Obj ->"Thank you, please restart programs"
                  |
:0040DA49 68B07D4400        push 00447DB0
:0040DA4E 8BCE              mov ecx, esi
:0040DA50 E882C80100        call 0042A2D7
:0040DA55 8B16              mov edx, dword ptr [esi]
:0040DA57 8BCE              mov ecx, esi
:0040DA59 FF92C4000000      call dword ptr [edx+000000C4]
:0040DA5F EB37              jmp 0040DA98

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:0040DA47(C)
|
* Possible StringData Ref from Data Obj ->"Invalid reg code"
                  |
:0040DA61 689C7D4400        push 00447D9C
:0040DA66 8BCE              mov ecx, esi
```

Si miramos vemos que delante de los mensajes que hay un salto controlado por la llamada (call 00417A2D), esta llamada es accedida en 23 ocasiones (damasiadas y todo), lo que tenemos seguro es que una de ellas es al introducir nuestros datos de registro y otra al reiniciar el programa, si le echamos un vistazo al código vemos que lo que necesiamos es que el call devuelva eax=0, pues entremos en él y miremos el modo de obligarla a que devuelva este valor en el registro eax, la llamada es la siguiente:

CALL 00417A2D:

```
* Referenced by a CALL at Addresses:
```

```
|:00401A12   , :00401A78   , :004021C9   , :004022C2   , :00404A08
|:00407AB9   , :00407B02   , :00407C7B   , :00407D37   , :004088FA
|:0040B085   , :0040BCE6   , :0040D7B9   , :0040D7F4   , :0040D993
|:0040DA36   , :0041481D   , :0041486C   , :00426F2E   , :0042B658
|:0042B66B   , :0042B67E   , :0042FA46
|
:00417A2D 55                    push ebp
:00417A2E 8BEC                  mov ebp, esp
:00417A30 833DDCC7440000        cmp dword ptr [0044C7DC], 00000000
:00417A37 53                    push ebx
:00417A38 56                    push esi
:00417A39 57                    push edi
:00417A3A 7512                  jne 00417A4E        /* salto BUENO */
:00417A3C FF750C                push [ebp+0C]
:00417A3F FF7508                push [ebp+08]
:00417A42 E809330000            call 0041AD50
:00417A47 59                    pop ecx
:00417A48 59                    pop ecx
:00417A49 E989000000            jmp 00417AD7

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00417A3A(C)
|
:00417A4E 6A19                  push 00000019
:00417A50 E878320000            call 0041ACCD
:00417A55 8B750C                mov esi, dword ptr [ebp+0C]
:00417A58 8B7D08                mov edi, dword ptr [ebp+08]
:00417A5B 59                    pop ecx

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00417AB7(U)
|
:00417A5C 660FB60F              movzx cx, byte ptr [edi]
:00417A60 0FB6C1                movzx eax, cl
:00417A63 47                    inc edi
:00417A64 894D0C                mov dword ptr [ebp+0C], ecx
:00417A67 F680E1C8440004        test byte ptr [eax+0044C8E1], 04
:00417A6E 7416                  je 00417A86
:00417A70 8A07                  mov al, byte ptr [edi]
:00417A72 84C0                  test al, al
:00417A74 7506                  jne 00417A7C
:00417A76 83650C00              and dword ptr [ebp+0C], 00000000
:00417A7A EB0A                  jmp 00417A86

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00417A74(C)
|
:00417A7C 33D2                  xor edx, edx
:00417A7E 47                    inc edi
:00417A7F 8AF1                  mov dh, cl
:00417A81 8AD0                  mov dl, al
:00417A83 89550C                mov dword ptr [ebp+0C], edx

* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:00417A6E(C), :00417A7A(U)
|
:00417A86 660FB61E              movzx bx, byte ptr [esi]
:00417A8A 0FB6C3                movzx eax, bl
:00417A8D 46                    inc esi
:00417A8E F680E1C8440004        test byte ptr [eax+0044C8E1], 04
:00417A95 7413                  je 00417AAA
:00417A97 8A06                  mov al, byte ptr [esi]
:00417A99 84C0                  test al, al
:00417A9B 7504                  jne 00417AA1
:00417A9D 33DB                  xor ebx, ebx
:00417A9F EB09                  jmp 00417AAA

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00417A9B(C)
|
:00417AA1 33C9                  xor ecx, ecx
```

```
:00417AA3 46                inc esi
:00417AA4 8AEB              mov ch, bl
:00417AA6 8AC8              mov cl, al
:00417AA8 8BD9              mov ebx, ecx

* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:00417A95(C), :00417A9F(U)
|
:00417AAA 66395D0C          cmp word ptr [ebp+0C], bx
:00417AAE 7509              jne 00417AB9    /* salto MALO */
:00417AB0 66837D0C00        cmp word ptr [ebp+0C], 0000
:00417AB5 7416              je 00417ACD     /* salto BUENO */
:00417AB7 EBA3              jmp 00417A5C

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00417AAE(C)
|
:00417AB9 6A19              push 00000019
:00417ABB E86E320000        call 0041AD2E
:00417AC0 663B5D0C          cmp bx, word ptr [ebp+0C]
:00417AC4 59                pop ecx
:00417AC5 1BC0              sbb eax, eax
:00417AC7 83E002            and eax, 00000002
:00417ACA 48                dec eax
:00417ACB EB0A              jmp 00417AD7

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:00417AB5(C)
|
:00417ACD 6A19              push 00000019
:00417ACF E85A320000        call 0041AD2E
:00417AD4 59                pop ecx
:00417AD5 33C0              xor eax, eax        /* pone eax a cero como queremos */

* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:00417A49(U), :00417ACB(U)
|
:00417AD7 5F                pop edi
:00417AD8 5E                pop esi
:00417AD9 5B                pop ebx
:00417ADA 5D                pop ebp
:00417ADB C3                ret                 /* retorno */
```

Los pasos a seguir para que devuelva eax=0 son: obligarle a saltar en :00417A3A, nopear en :00417AAE y obligarle a saltar en :00417AB5.

Así pues los cambio a realizar son:

```
:00417A3A 7512      jne 00417A4E    POR:
:00417A3A EB12      jmp 00417A4E

:00417AAE 7509      jne 00417AB9    POR:
:00417AAE 9090      nop (2 veces)

:00417AB5 7416      je 00417ACD     POR :
:00417AB5 EB16      jmp 00417ACD
```

Aplicamos los cambios con iun editor hexadecimal y ya tenemos nuestra versión totalmente registrada, ya nos damos cuenta que entramos sin ver la nag de entrada, buena señal, y así con el resto del programa ;D

Y como siempre decir que espero que os haya servido de ayuda, pronto volveremos con más y mejores tutoriales. Para cualquier duda, crítica o sugerencia me encontraréis en:

syxe00@hotmail.com
syxe00@yahoo.es